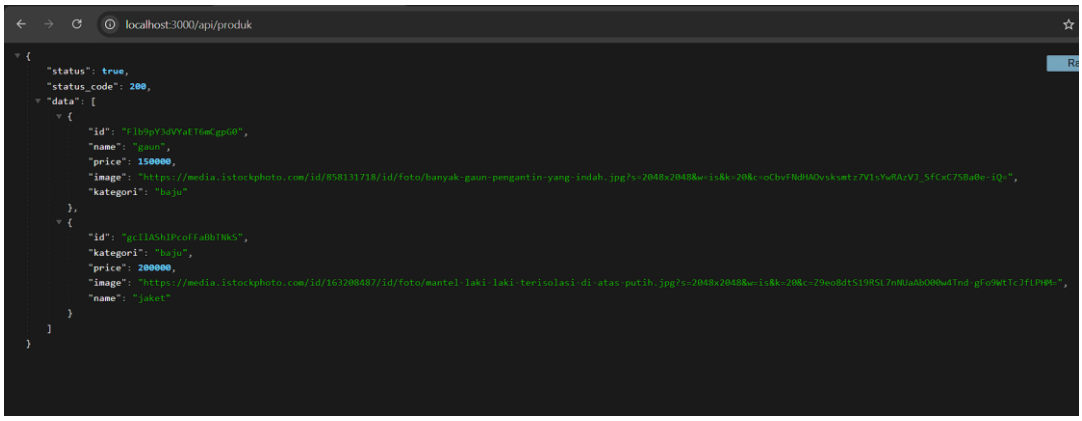
KEMENTERIAN PENDIDIKAN, KEBUDAYAAN, RISET, DAN TEKNOLOGI
POLITEKNIK NEGERI MALANG
JURUSAN TEKNOLOGI INFORMASI
Jl. Soekarno Hatta No. 9, Jatimulyo, Lowokwaru, Malang 65141
Telp. (0341) 404424 – 404425, Fax (0341) 404420
https://jti.polinema.ac.id

Mata Kuliah	:	**PEMROGRAMAN BERBASIS FRAMEWORK (PBF)**
Program Studi	:	D4 – Teknik Informatika
Semester	:	6

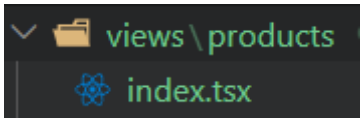
Kelas	:	TI-3B
NIM	:	2341720164
Nama	:	Raudhil Firdaus Naufal
Jobsheet Ke-	:	8

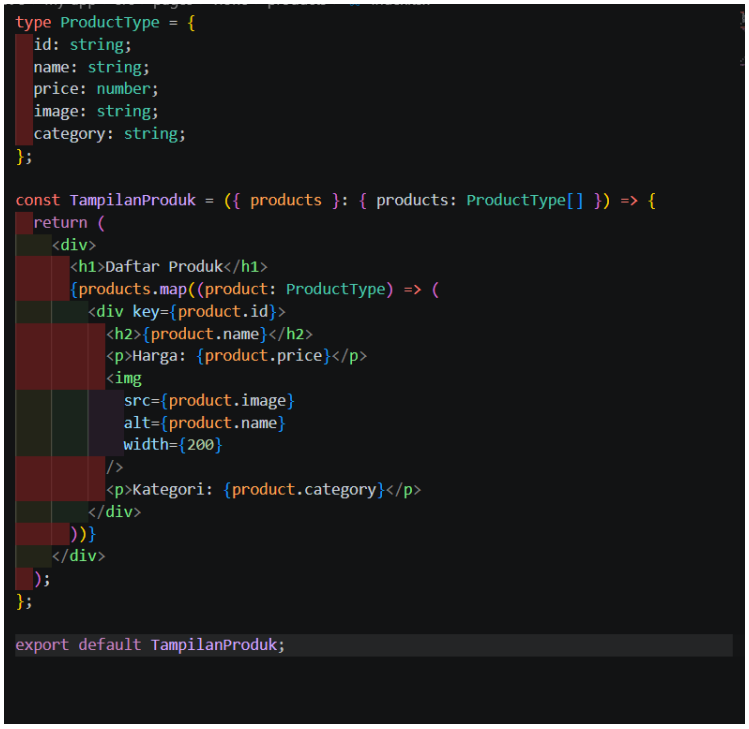
Laporan Jobsheet

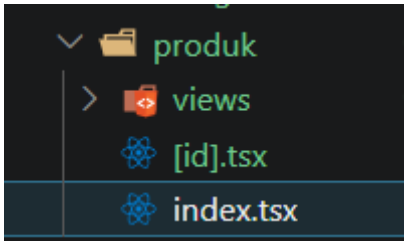
Langkah 1 - Setup Data Produk

Langkah	Jawaban/Deskripsi
1	Siapkan project Next.js.
2	Buat endpoint API /api/products.
3	Pastikan data memiliki: o id o name o category o price o image
4	jalankan browser http://localhost:3000/api/produk  Berhasil menambahkan field baru

Langkah 2 - Implementasi CSR dengan useEffect

Langkah	Jawaban/Deskripsi
1	Membuat file index.tsx pada folder views/products 

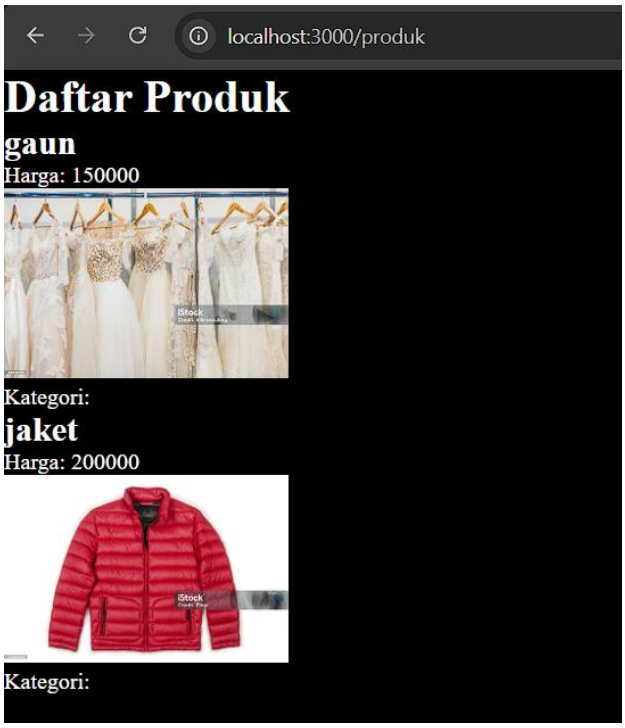
2	Modifikasi index.tsx
	 
	`type ProductType = {`
	`  id: string;`
	`  name: string;`
	`  price: number;`
	`  image: string;`
	`  category: string;`
	`};`
	
	`const TampilanProduk = ({ products }: { products: ProductType[] }) => {`
	`  return (`
	`    <div>`
	`      <h1>Daftar Produk</h1>`
	`      {products.map((product: ProductType) => (`
	`        <div key={product.id}>`
	`          <h2>{product.name}</h2>`
	`          <p>Harga: {product.price}</p>`
	`          <img`
	`            src={product.image}`
	`            alt={product.name}`
	`            width={200}`
	`          />`
	`          <p>Kategori: {product.category}</p>`
	`        </div>`
	`      ))}`
	`    </div>`
	`  );`
	`};`
	
	`export default TampilanProduk;`

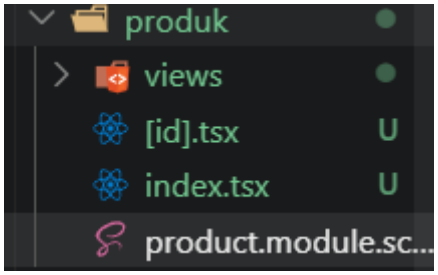
3	Buka file index.tsx pada pages/produk/ 
4	

Modifikasi index.tsx pada pages/produk/

```
import { useEffect, useState } from "react";
import { useRouter } from "next/router";
import TampilanProduk from "../views/products";

type ProductType = {

const Kategori = () => {
  // const [isLogin, setIsLogin] = useState(false);
  // const { push } = useRouter();
  const [products, setProducts] = useState<any[]>([]);

  // console.log("products", products);

  // useEffect(() => {
  //   if (!isLogin) {
  //     push("/auth/login");
  //   }
  // }, [isLogin]);

  useEffect(() => {
    fetch("/api/produk")
      .then((response) => response.json())
      .then((responsedata) => {
        setProducts(responsedata.data);
        // console.log("Data produk:", responsedata.data);
      })
      .catch((error) => {
        console.error("Error fetching produk:", error);
      });
  }, []);

  return (
    <div>
      <TampilanProduk products={products} />
    </div>
  );
};

export default Kategori;
```

5 Jalankan browser http://localhost:3000/produk



6	Pada folder produk buat file produk.modules.scss 

7	Modifikasi produk.modules.scss
	`.product { width: 100%; padding: 0 5%; text-align: center; font-size: 32px; font-weight: bold; margin-bottom: 16px; }`
	`.content { display: flex; flex-wrap: wrap; gap: 20px; }`
	`.item { width: 100%; }`
	`.image { width: 100%; height: 200px; object-fit: cover; border-radius: 4px; margin-bottom: 12px; }`
	`.name { font-size: 18px; font-weight: 600; margin-bottom: 8px; }`
	`.price { font-size: 16px; color: #ff5722; font-weight: bold; margin-bottom: 8px; }`
	`.category { font-size: 14px; color: #888888; margin-bottom: 8px; }`

8	Modifikasi Pada file index.tsx pada folder pages/views/product

```
index.tsx ...\products U ×   index.tsx Jobsheet 8\my-app\src\pages\produk U   product.module.sc
Jobsheet 8 > my-app > src > pages > views > products > index.tsx > default
1  import styles from "../product/product.module.scss";
2
3  type ProductType = {
4    id: string;
5    name: string;
6    price: number;
7    image: string;
8    category: string;
9  };
10
11 const TampilanProduk = ({ products }: { products: ProductType[] }) => {
12   return (
13     <div className={styles.product}>
14       <h1 className={styles.produk_title}>Daftar Produk</h1>
15
16       <div className={styles.produk_content}>
17         {products.map((product: ProductType) => (
18           <div
19             key={product.id}
20             className={styles.produk_content_item}
21           >
22             <img
23               src={product.image}
24               alt={product.name}
25               width={200}
26             />
27
28             <h4 className={styles.produk_content_item_name}>
29               {product.name}
30             </h4>
31
32             <p className={styles.produk_content_item_category}>
33               {product.category}
34             </p>
35
36             <p className={styles.produk_content_item_price}>
37               Rp {product.price.toLocaleString()}
38             </p>
39           </div>
40         ))}
41       </div>
42     </div>
43   );
44 };
45
46 export default TampilanProduk;
```

9	Jalankan Browser

Berhasil menampilkan gambar pada browser

Langkah 3 – Implementasi Skeleton Loading

Langkah	Jawaban/Deskripsi
1	Modfikasi file index.tsx pada folder views/product/index.tsx

```
import styles from "../../produk/product.module.
scss";

type ProductType = {
  id: string;
  name: string;
  price: number;
  image: string;
  category: string;
};

const TampilanProduk = ({ products }: {
products: ProductType[] }) => {
  return (
    <div className={styles.product}>
      <h1 className={styles.produk_title}>Daftar
      Produk</h1>

      <div className={styles.produk_content}>
        {/* Skeleton Loader (ditampilkan saat
        loading) */}
        <div className={styles.
        produk_content_skeleton}>
          <div className={styles.
          produk_content_skeleton_image}></div>
          <div className={styles.
          produk_content_skeleton_name}></div>
          <div className={styles.
          produk_content_skeleton_category}></
          div>
          <div className={styles.
          produk_content_skeleton_price}></div>
        </div>

        {/* Daftar Produk */}
        {products.map((product: ProductType) => (
          <div
            key={product.id}
            className={styles.
            produk_content_item}
          >
            <img
              src={product.image}
              alt={product.name}
              width={200}
            />

            <h4 className={styles.
            produk_content_item_name}>
              {product.name}
            </h4>
```

2	Modifikasi file product.module.scss

```scss
.product {
  width: 100%;
  padding: 0 5%;
  text-align: center;
  font-size: 32px;
  font-weight: bold;
  margin-bottom: 16px;
}

.produk_content {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
  justify-content: center;
}

.produk_content_skeleton {
  width: 280px;
  border: 1px solid #eee;
  border-radius: 8px;
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
  display: flex;
  flex-direction: column;
  align-items: center;
  animation: identifier 1.5s infinite
  ease-in-out;
}

.produk_content_skeleton_image {
  width: 100%;
  height: 200px;
  background-color: #e0e0e0;
  border-radius: 4px;
  margin-bottom: 12px;
}

.produk_content_skeleton_name {
  width: 80%;
  height: 20px;
  background-color: #e0e0e0;
  border-radius: 4px;
  margin-bottom: 8px;
}

.produk_content_skeleton_category {
  width: 60%;
  height: 16px;
  background-color: #e0e0e0;
  border-radius: 4px;
  margin-bottom: 8px;
}
```

3	Jalankan browser maka akan muncul skeleton yang terdapat animasi berkedip
4	Modifikasi pada index.tsx pada folder views/product/index.tsx

```tsx
import styles from "../product/product.module.scss";

type ProductType = {
  id: string;
  name: string;
  price: number;
  image: string;
  category: string;
};

const TampilanProduk = ({ products }: { products: ProductType[] }) => {
  return (
    <div className={styles.product}>
      <h1 className={styles.produk_title}>Daftar Produk</h1>

      <div className={styles.produk_content}>
        {products.length > 0 ? (
          // Tampilkan produk asli jika ada data
          products.map((product: ProductType) => (
            <div
              key={product.id}
              className={styles.produk_content_item}
            >
              <img
                className={styles.produk_content_item_image}
                src={product.image}
                alt={product.name}
                width={200}
              />

              <h4 className={styles.produk_content_item_name}>
                {product.name}
              </h4>

              <p className={styles.produk_content_item_category}>
                {product.category}
              </p>

              <p className={styles.produk_content_item_price}>
                Rp {product.price.toLocaleString()}
              </p>
            </div>
          ))
        ) : (
          // Tampilkan skeleton loader jika belum ada data
          <div className={styles.produk_content_skeleton}>
            <div className={styles.produk_content_skeleton_image}></div>
            <div className={styles.produk_content_skeleton_name}></div>
            <div className={styles.produk_content_skeleton_category}></div>
            <div className={styles.produk_content_skeleton_price}></div>
          </div>
        )}
```

5	Jalankan browser

Langkah 4 – Implementasi SWR

Langkah	Jawaban/Deskripsi

1	Install swr
	`PS D:\Matkul\Semester 6\FrameWork\praktikum\Pemrograman-Berbasis-Framework\Jobsheet 8\my-app> npm install swr`
	
	`added 67 packages, and audited 496 packages in 6s`
	
	`179 packages are looking for funding`
	`  run `npm fund` for details`
	
	`4 vulnerabilities (2 moderate, 2 high)`
	
	`To address issues that do not require attention, run:`
	`  npm audit fix`
	
	`To address all issues, run:`
	`  npm audit fix --force`
	
	`Run `npm audit` for details.`

2	Buka dan modifkasi file index.tsx pada folder pages/product/

```
import { useRouter } from "next/router";
import { useEffect, useState } from "react";
import TampilanProduk from "../views/products";
import useSWR from "swr";

const fetcher = (url: string) => fetch(url).then((res) => res.json());

const Kategori = () => {
  const [isLogin, setIsLogin] = useState(false);
  const [products, setProducts] = useState([]);

  // console.log("products", products);

  const { data, error, isLoading } = useSWR("/api/produk", fetcher);

  // console.log("data:", data);
  // console.log("error:", error);
  // console.log("isLoading:", isLoading);

  return (
    <div>
      <TampilanProduk
        products={isLoading ? [] : data?.data || []}
      />
    </div>
  );
};

export default Kategori;
```

3 Agar terlihat lebih rapi

```
Jobsheet 8 > my-app > src > pages > utils > swr > TS fetcher.ts > default
1  const fetcher = (url: string) => fetch(url).then((res) => res.json());
2
3  export default fetcher;
```

```
import { useRouter } from "next/router";
import { useEffect, useState } from "react";
import TampilanProduk from "../views/products";
import useSWR from "swr";
import fetcher from "../utils/swr/fetcher";

const Kategori = () => {
  // const [isLogin, setIsLogin] = useState(false);
  // const { push } = useRouter();
  // const [products, setProducts] = useState([]);

  // console.log("products", products);

  const { data, error, isLoading } = useSWR("/api/produk", fetcher);

  // Cek data, error, dan isLoading
  // console.log("data:", data);
  // console.log("error:", error);
  // console.log("isLoading:", isLoading);

  return (
    <div>
      <TampilanProduk
        products={isLoading ? [] : (data?.data || [])}
      />
    </div>
  );
};

export default Kategori;
```

	Perbandingan Use Effect Manual: Fetch data dengan useeffect + usestate SWR: Library untuk fetch data agar lebih simpel

D. Tugas Praktikum

Langkah	Jawaban/Deskripsi
1	Jelaskan perbedaan: Client Side Rendering: Rendering dilakukan di browser setelah halaman dimuat menggunakan JavaScript. Server Side Rendering: Rendering dilakukan di server setiap ada permintaan dari user, lalu HTML dikirim ke browser. Static Site Generation: Rendering dilakukan saat proses build, menghasilkan file HTML statis yang siap disajikan.
2	Buat halaman produk dengan: Skeleton loading Animasi
3	

Refactor kode dari useEffect menjadi SWR

```
import { useRouter } from "next/router";
import { useEffect, useState } from "react";
import TampilanProduk from "../views/products";
import useSWR from "swr";
import fetcher from "../utils/swr/fetcher";

const Kategori = () => {
  // const [isLogin, setIsLogin] = useState
  (false);
  // const { push } = useRouter();
  // const [products, setProducts] = useState
  ([]);

  // console.log("products", products);

  const { data, error, isLoading } = useSWR("/
  api/produk", fetcher);

  // Cek data, error, dan isLoading
  // console.log("data:", data);
  // console.log("error:", error);
  // console.log("isLoading:", isLoading);

  return (
    <div>
      <TampilanProduk
        products={data?.data || []}
        isLoading={isLoading}
        error={error}
      />
    </div>
  );
};

export default Kategori;
```